

Global Solution FIAP 2026-1 — Sistema Inteligente de Monitoramento Espacial

Relatório Técnico

Junho de 2026

Global Solution FIAP 2026-1 — Sistema Inteligente de Monitoramento Espacial

Equipe: Gabriel Carmona Bittencourt (RM569239) · Iúri Leão de Almeida (RM570215) · Márcio Francisco dos Santos Júnior (RM570758) · Maria Sophia Domingues dos Santos (RM571209)

Atividade Integradora — Ciência da Computação (online), FIAP, 2026

Repositório: <https://github.com/iurileo-hub/global-solution>

Resumo

Este trabalho apresenta um sistema inteligente de monitoramento da missão espacial da colônia Aurora Siger em Marte. O sistema é construído em duas camadas deliberadamente separadas: um **motor científico determinístico** reaproveitado da Fase 3 da atividade integradora (simulação de 168 horas, geração/consumo físicos, árvores hierárquicas, OLS à mão) e uma **camada de monitoramento nova**, desenvolvida para a Global Solution, que adiciona estruturas de dados explícitas (fila, pilha, matriz), detecção de inconsistências em telemetria, avaliação lógica de alertas e previsão de reserva energética. O ponto de entrada `src/sistema.py` lê o CSV canônico `data/dados.csv`, organiza os dados em estruturas, aplica regras booleanas e OLS, e imprime um relatório operacional organizado em painéis, com cor adaptativa ao terminal e codificação segura (Seção 7). Toda a cadeia de execução respeita a restrição autoimposta de `stdlib-only` — sem `numpy`, `sklearn` ou qualquer dependência externa no caminho de runtime.

1. Introdução

O projeto Aurora Siger acompanha a missão em três atos: **decolagem** (Fase 1 — telemetria e decisão Go/No-Go), **pouso** (Fase 2 — autorização e estabilização) e **operação** (Fase 3 — energia e decisão contínua). A Global Solution acrescenta um quarto ato: o **monitoramento inteligente** dessa operação, integrando de forma explícita as estruturas e conceitos de todas as fases anteriores.

A decisão de projeto central foi **vendorizar** o motor da Fase 3 — isto é, incorporar uma cópia do código ao próprio repositório — em vez de tomá-lo como dependência instalável. O diretório `src/engine/` contém uma cópia fiel dos 15 módulos de `aurora_siger.operations`, com apenas os imports ajustados para o novo namespace (`from engine.X`). Essa escolha preserva a física já validada da Fase 3 e deixa o código legível, rastreável e auditável dentro do próprio repositório. A camada nova, `src/monitor/`, foi desenvolvida de forma incremental e cobre os requisitos do §7 ao §14 do enunciado da Global Solution.

2. Análise dos dados de telemetria

2.1 O conjunto de dados canônico

O arquivo `data/dados.csv` contém **168 registros** — um por hora de operação, cobrindo 7 sóis marcianos (cada sol tem 24 horas no modelo). Cada registro inclui 24 campos: passo (`step`), posição temporal (`sol`, `hour`), condições climáticas (`temperature_c`, `wind_ms`, `tau`, `storm`), geração por fonte e total (`solar_kw`, `wind_kw`, `nuclear_kw`, `generation_kw`), consumo (`consumption_kw`), estado da bateria (`battery_kwh`, `battery_pct`), contagem de falhas (`broken_count`), nível de energia classificado (`energy_level`), tendência OLS (`slope`, `predicted_delta`) e o status binário de 6 módulos monitorados (`mod1_ok`, `mod2_ok`, `mod3_ok`, `mod7_ok`, `mod6_ok`, `mod8_ok` — os quatro vitais e dois de sustento).

O CSV é gerado deterministicamente por `telemetry_io.export_run(seed=42)`: toda fonte de aleatoriedade (clima, falhas de módulos) passa pelo mesmo gerador congruencial linear (LCG) injetado no estado da simulação. A mesma seed produz históricos byte a byte idênticos, garantindo reprodutibilidade total da telemetria.

Os números consolidados da execução canônica:

Métrica	Valor
Horas simuladas	168 (7 sóis)
Geração média	86,5 kW
Consumo médio	85,2 kW
Bateria final	328,1 / 500,0 kWh (65,6%)
Horas com tempestade	63
Eventos no log	30 (3 falhas, 3 auto-reparos, 1 clima, 23 energia)
Eventos críticos totais	65

2.2 A inconsistência proposital

A função `inject_inconsistency` planta **uma** inconsistência documentada no passo 50: `battery_pct = 142.0`, violando o invariante físico $[0, 100]\%$. O propósito é demonstrar que o sistema detecta e reporta anomalias em vez de consumi-las silenciosamente.

`detect_inconsistencies` valida três classes de invariantes físicos:

1. **Percentual de bateria** fora de $[0, 100]\%$ — captura a inconsistência plantada.
2. **Energia negativa** — geração ou consumo abaixo de zero seria fisicamente impossível.
3. **Nível de tempestade inválido** — valor fora do conjunto `{clear, light, moderate, severe}` indicaria corrupção de dado.

Na execução canônica, apenas o passo 50 dispara a regra 1, confirmando que o detector é sensível o suficiente para encontrar a anomalia plantada e robusto o suficiente para não produzir falsos positivos nos demais 167 passos.

3. Estruturas de dados

A escolha de cada estrutura foi guiada pelo padrão de acesso que o problema exige. A tabela abaixo resume as seis estruturas empregadas.

Estrutura	Módulo	Por que esta estrutura
Lista (séries horárias)	<code>engine/</code> e <code>monitor/telemetry_io.py</code>	Acesso indexado por hora; iteração sequencial
Dicionário (módulos, snapshots)	<code>engine/modules.py</code> , <code>sistema.py</code>	Lookup $O(1)$ por ID ou nome de campo
Árvore N-ária (Node)	<code>engine/tree.py</code> , <code>engine/hierarchies.py</code>	Hierarquia Vital > Sustento > Expansão para load shedding
Fila (Queue / AlertQueue)	<code>monitor/structures.py</code> , <code>monitor/alerts.py</code>	Ordenação FIFO dentro de cada raia de severidade

Estrutura	Módulo	Por que esta estrutura
Pilha (<code>Stack</code> / <code>CriticalEventStack</code>)	<code>monitor/structures.py</code> , <code>monitor/alerts.py</code>	Acesso LIFO aos eventos críticos mais recentes
Matriz (<code>ReadingsMatrix</code>)	<code>monitor/matrix.py</code>	Indexação bidimensional hora × variável

3.1 Árvore N-ária de criticidade

`engine/hierarchies.py` constrói duas árvores N-árias sobre a mesma lista plana de 13 módulos, usando a classe genérica `Node` de `engine/tree.py`. A árvore de criticidade — a mais relevante para o monitoramento — tem três camadas:

- **Vital** (ids 1, 2, 3, 7 — Command and Control, Life Support, Habitat, Medical): nunca desligam; sua quebra é condição suficiente para alerta CRÍTICO.
- **Sustento** (ids 4, 5, 6, 8, 10, 13): geração e suporte secundário; rebaixados de modo quando a oferta é insuficiente para o nível acima.
- **Expansão** (ids 9, 11, 12 — Logística, Oficina, Lab Científico): primeiro a ceder em situação de déficit.

Como as duas árvores referenciam os mesmos dicionários de módulo, alterar `module["current_mode"]` é imediatamente visível em qualquer das árvores — sem cópia nem sincronização manual. O alocador percorre a árvore de criticidade de baixo para cima para decidir quais módulos rebaixar.

3.2 Fila e pilha à mão

`monitor/structures.py` implementa `Queue[T]` (FIFO) e `Stack[T]` (LIFO) sobre uma `list` Python, sem usar `collections.deque`. A decisão foi deliberada: a rubrica exige que a estrutura seja *visível e justificada*; esconder a implementação atrás de um tipo de biblioteca impede a inspeção da semântica.

`AlertQueue` (em `monitor/alerts.py`) compõe três `Queues` — uma por nível de severidade (CRÍTICO, ALERTA, NORMAL) — e drena na ordem de prioridade, preservando FIFO dentro de cada raia. `CriticalEventStack` envolve a `Stack` genérica e expõe `push_event` e `recent(n)`, devolvendo os n eventos mais recentes em ordem do mais novo para o mais antigo — exatamente o acesso LIFO que uma pilha oferece de forma natural.

3.3 Matriz de leituras

`ReadingsMatrix` representa as 168 horas como uma lista-de-listas [$hora \times var$], sem numpy. O método de classe `from_telemetry` recebe a lista de registros e os nomes das variáveis desejadas, construindo a matriz por compreensão. Os métodos `get(hour, var)` e `column(var)` permitem acesso pontual e extração de série temporal, respectivamente. Na execução de referência a matriz cobre 5 variáveis: `temperature_c`, `wind_ms`, `generation_kw`, `consumption_kw`, `battery_pct`.

4. Lógica de decisão e regras de alerta

4.1 Expressão booleana principal

O diagnóstico crítico da missão é governado pela expressão booleana:

$$CRITICO = (consumo > geracao) \wedge (bat_baixa \vee vital_falho) \wedge \neg em_recuperacao$$

onde $em_recuperacao \equiv slope > 0$ (a tendência OLS dos deltas de energia aponta para cima). A lógica codifica a intuição de que um déficit instantâneo sozinho não constitui emergência: a bateria absorve o vale noturno, e se a tendência for positiva o sistema está se recuperando por conta própria. Apenas a combinação dos três fatores — déficit atual, risco vital e sem recuperação — justifica acionar o alerta CRÍTICO.

Em `monitor/alerts.py`, a expressão é implementada linha a linha com `and`, `or`, `not` explícitos:

```
if (consumption > generation) and (low_battery or vital_broken) and (not in_recovery):  
    # alerta CRÍTICO: ENERGY_DEFICIT
```

Isso reflete diretamente a álgebra booleana, tornando a regra inspecionável e testável de forma isolada.

4.2 As quatro regras de alerta

`evaluate_alerts(snapshot)` é uma **função pura** — recebe um dicionário instantâneo e devolve uma lista de objetos `Alert`, sem efeito colateral e sem depender de estado externo. As quatro regras são:

1. **CRÍTICO / ENERGY_DEFICIT** — expressão booleana principal acima.
2. **ALERTA / LOW_ENERGY** — bateria abaixo de 40% *ou* slope OLS $\leq -2,0$ kW/passo. Captura tanto a situação presente quanto a tendência negativa acentuada.
3. **ALERTA / CLIMATE** — tempestade de nível *moderate* ou *severe* e nenhum módulo vital quebrado concorrentemente (para não duplicar a gravidade do alerta `VITAL_FAILURE`).
4. **CRÍTICO / VITAL_FAILURE** — qualquer módulo Vital (ids 1, 2, 3, 7) fora de operação.

Se nenhuma das quatro regras disparar, a função retorna um único alerta `NORMAL/OK`, garantindo que o relatório sempre tenha ao menos uma linha de diagnóstico.

Vale registrar, por transparência, um limite da cobertura dos dados: na execução canônica (seed 42) as 63 horas de tempestade são todas de nível *light*, abaixo do gatilho *moderate/severe* da regra 3 — portanto o alerta `CLIMATE` permanece dormente nesta telemetria específica. A regra existe e é exercitada por testes unitários que injetam cenários *moderate/severe*; ela apenas não é acionada pela trajetória canônica.

4.3 Separação entre regra e apresentação

A separação entre `evaluate_alerts` (pura) e `AlertQueue/CriticalEventStack` (apresentação) não é meramente estética: ela garante que a lógica de diagnóstico possa ser testada sem instanciar filas, que a fila possa ser testada sem regras reais, e que a pilha de histórico seja construída independentemente dos alertas correntes. Em `sistema.py`, os dois fluxos são explícitos: (a) `evaluate_alerts(snapshot_final)` alimenta a fila para o diagnóstico do último passo; (b) um loop sobre toda a telemetria alimenta a pilha de histórico crítico.

4.4 Déficit instantâneo não é emergência

Um resultado revelador da execução canônica: em **93 das 168 horas** a geração instantânea ficou abaixo do consumo, mas o alerta `CRÍTICO` de `ENERGY_DEFICIT` só dispara quando o déficit coincide com bateria baixa *ou* módulo vital quebrado — e, ainda assim, sem tendência de recuperação. À noite, o solar zera e apenas o nuclear sustenta a base; o consumo instantâneo supera a geração, mas a bateria absorve o vale e recarrega ao longo do dia. Decidir só pelo déficit instantâneo seria reagir a um falso alarme dezenas de vezes por missão.

5. Modelo de previsão

5.1 OLS à mão, forma fechada

`engine/prediction.py` implementa a regressão linear por mínimos quadrados na forma fechada, sem `numpy`:

$$a = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}, \quad b = \bar{y} - a\bar{x}$$

A função `linear_regression(xs, ys)` devolve **(a, b)** usando apenas `sum()`, `len()` e um laço `for`. Nenhuma dependência externa: o §12 do enunciado até permite bibliotecas como `numpy`, desde que a lógica seja construída pelos alunos — a opção aqui foi pela leitura mais estrita, com o caminho de execução inteiro em `stdlib`.

5.2 Dois usos da mesma função

O estimador é reutilizado em dois contextos distintos:

1. **Tendência de energia** (`fit_energy_trend`): treina sobre a série de deltas *geracao* — *consumo* na janela recente e devolve o slope e a projeção do próximo passo. O slope positivo é o critério de “em recuperação” que entra na expressão booleana da Seção 4.1.
2. **Reserva futura** (`predict_next_reserve` em `sistema.py`): treina sobre `battery_pct` vs `step` na janela de 12 passos finais e extrapola a reserva no próximo ciclo. Na execução canônica, o slope final é $-3,289\%$ /passo e a reserva prevista é 68,0% — acima do limiar de 40%, portanto sem acionar recomendação preventiva nesse passo final.

5.3 A previsão influencia a decisão

A integração entre previsão e decisão é unidirecional e explícita: o slope calculado por OLS entra como campo `slope` no snapshot passado a `evaluate_alerts`. A regra 2 (`LOW_ENERGY`) dispara se o slope for $\leq -2,0$, mesmo que a bateria atual ainda esteja acima de 40%. Isso significa que a OLS não é apenas um relatório descritivo — ela **atua** como sensor de tendência que pode antecipar o alerta antes de a bateria cruzar o limiar crítico.

5.4 Por que OLS de forma fechada, não gradiente descendente

A solução fechada é exata (não itera), não tem taxa de aprendizado para calibrar, não diverge em séries de curta janela e não exige clamp anti-explosão. Para janelas de poucas dezenas de pontos — como as 12 amostras da janela de previsão — o custo computacional é irrelevante e o ganho em auditabilidade é alto: cada coeficiente é calculável à mão a partir dos dados, tornando o modelo completamente transparente.

6. Decisões técnicas

6.1 Vendoring do motor da Fase 3

O diretório `src/engine/` contém os 15 módulos de `aurora_siger.operations` copiados fielmente, com um único ajuste: todos os imports `from aurora_siger.operations.X` foram reescritos para `from engine.X`. Nenhuma lógica do motor foi alterada no transplante.

A alternativa seria instalar o pacote da Fase 3 como dependência pip. O vendoring foi preferido porque: (a) torna o repositório autossuficiente, sem nenhuma dependência externa para rodar; (b) isola a Global Solution de alterações futuras no repositório de origem; (c) mantém o código do motor visível e navegável sem precisar seguir imports para um pacote externo; (d) atende diretamente ao §12 do enunciado, que pede o uso dos conceitos desenvolvidos nas fases 1, 2 e 3.

6.2 Runtime stdlib-only

Todo o caminho de execução de `sistema.py` usa exclusivamente a biblioteca padrão do Python: `csv`, `os`, `sys`, `re`, `collections.Counter`, `dataclasses`. O projeto não tem nenhuma dependência de terceiros — nem em runtime, nem para instalação. Isso satisfaz o §12 do enunciado e garante que o sistema rode em qualquer ambiente Python 3.10 ou superior sem instalação adicional.

6.3 `evaluate_alerts` como função pura

Manter `evaluate_alerts` como função pura (snapshot dict $\rightarrow$ list[Alert], sem efeito colateral) foi uma decisão deliberada de testabilidade e auditabilidade. Com ela: (a) cada regra pode ser exercitada isoladamente com um dict literal, sem montar o sistema inteiro; (b) a fila e a pilha podem ser verificadas independentemente com objetos `Alert` criados diretamente; (c) o sistema pode aplicar a mesma função a cada um dos 168 passos históricos para construir a pilha de eventos críticos sem precisar rearrquitetar o estado.

6.4 Determinismo por LCG com seed

Toda aleatoriedade da simulação — variações climáticas, tempestades, falhas de módulos e auto-reparos — passa pelo gerador congruencial linear (LCG) injetado em `state["rng"]`. O estado inicial do LCG é determinado exclusivamente pela seed passada a `init_simulation(seed)`. Não há chamadas ao módulo `random` da biblioteca padrão nem a `time.time()`. Duas chamadas a `export_run(seed=42)` produzem CSVs byte a byte idênticos — garantia de reprodutibilidade total da telemetria canônica.

7. Camada de apresentação

A saída do sistema não é um despejo de texto plano: é um relatório em painéis com moldura, cor e mini-gráficos, projetado para que o leitor — operador ou avaliador — perceba a cobertura dos requisitos sem precisar garimpá-la. A camada é isolada da lógica em três módulos, concretizando o princípio de separar regra de apresentação das Seções 4.3 e 6.3:

- **monitor/render.py** — primitivas puras de renderização (molduras, barras, *sparklines*, colorização), sem nenhum conhecimento de domínio.
- **monitor/rubric.py** — os cinco critérios técnicos da rubrica como **fonte única de verdade**, consumida tanto pelas etiquetas de cada seção quanto pelo painel final de cobertura. Um teste *anti-drift* garante que toda etiqueta exibida corresponda a um critério registrado, impossibilitando que a cobertura anunciada divirja das seções.
- **monitor/report.py** — compõe as seções a partir dos dados, sem imprimir; **sistema.py** faz a única chamada a `print`.

7.1 Robustez: cor adaptativa e codificação segura

A apresentação se adapta ao ambiente sem nunca interromper a execução — protegendo diretamente o critério “código executa sem erros”:

- **Cor** apenas quando a saída é um terminal interativo; ao redirecionar a saída (`> arquivo.txt`) ou definir `NO_COLOR`, degrada automaticamente para texto puro, sem sequências de escape contaminando o arquivo.
- **Glifos Unicode** (caixas, barras, *sparklines*) caem para equivalentes ASCII quando o terminal não os suporta (ex.: console Windows cp1252), e uma rede de segurança (`stdout.reconfigure(errors="replace")`) assegura que nenhum caractere fora da codificação derrube o programa.

7.2 A expressão booleana, acesa

O diferencial pedagógico do relatório é tornar a álgebra da Seção 4.1 **visível em execução**: cada termo aparece com seu valor-verdade e o resultado é recalculado na tela. Não é uma descrição da regra — é a própria regra avaliada sobre o passo corrente:

```
┌─── DIAGNÓSTICO ────┐ ──── lógica e regras ────┐
│ passo              167 · sol 6 · 23h          │
│ bateria ██████████░░░░░░░░░░░░░░░░░░░░░░ 65.6% [ NORMAL ] │
│ geração 81.0 kW   consumo 107.5 kW          │
│ vitais (1,2,3,7) ●●●●●                      │
│ CRÍTICO = (consumo>geração) ∧ (bat_baixa v vital_quebr) ∧ ¬recuperação │
│ consumo>geração=V   bat_baixa=F   vital_quebr=F   ¬recuperação=F   │
│ ⇒ CRÍTICO = F                                           │
└──────────────────────────────────────────────────────────────────────────┘
```

7.3 Rastreabilidade dos requisitos

O relatório encerra com um painel que mapeia cada critério técnico da rubrica à evidência concreta exibida acima — rastreabilidade de requisitos, não auto-avaliação:

```
┌─── COBERTURA DE REQUISITOS ────┐ ──── rastreabilidade ────┐
│ ✓ Interpretação de dados  anomalia detectada + matriz de leituras │
│ ✓ Estruturas de dados     fila.pilha.dict.árvore.matriz          │
│ ✓ Lógica e regras         AND/OR/NOT + expressão booleana avaliada │
│ ✓ Análise e previsão      OLS ⇒ recomendação disparada          │
│ ✓ Código Python          stdlib · funções puras · sem dependências │
└──────────────────────────────────────────────────────────────────────────┘
```

Vídeo e documentação ficam deliberadamente fora do painel: a interface de linha de comando atesta apenas aquilo que demonstra em tempo de execução.

8. Arquitetura do sistema

O fluxo de dados do sistema segue uma pipeline linear e sem ciclos:

```
init_simulation(seed=42)
|
step() x 168
|
export_run() -> data/dados.csv (+ inconsistencia plantada no passo 50)
|
read_telemetry() -> list[dict]
|
+-----+
| ReadingsMatrix [168 x 5]   (bidimensional) |
| detect_inconsistencies()  (invariantes)    |
| build_event_log()         (eventos)         |
| predict_next_reserve()    (OLS)             |
| evaluate_alerts(snapshot) (regras puras)    |
| -> AlertQueue CRITICO->ALERTA->NORMAL      |
| evaluate_alerts() x 168 -> CriticalStack    |
+-----+
|
compose_report() -> render/ + rubric/ (apresentacao)
|
Relatorio rico em paineis (stdout; cor e Unicode adaptativos)
```

A separação entre `engine/` (física, simulação, OLS) e `monitor/` (estruturas, alertas, I/O e apresentação) reflete a separação arquitetural entre o que existia na Fase 3 e o que foi construído para a Global Solution. As duas camadas se comunicam exclusivamente pelo CSV e pelos dicionários de snapshot — sem acoplamento direto entre módulos das duas camadas.

9. Conclusão

A Global Solution materializa a integração pedida pelo enunciado: reaproveitou o motor científico da Fase 3 (determinismo, OLS, árvores, controle de carga) e sobre ele construiu uma camada de monitoramento que adiciona detecção de inconsistências, estruturas explícitas (fila, pilha, matriz) e regras lógicas auditáveis.

A lição de engenharia central é que o preditivo e o reativo não se substituem — se estratificam. A OLS vigia a tendência e pode antecipar o alerta antes de a bateria cruzar o limiar; a expressão booleana evita falsos alarmes ao exigir a conjunção de déficit, risco vital e ausência de recuperação. Um sistema que apenas reage sobrevive a cada hora; um que também prevê começa a planejar o próximo sol.

Referências

- Repositório Global Solution: <https://github.com/iurileo-hub/global-solution>
- Motor da Fase 3 (origem do vendoring): <https://github.com/iurileo-hub/FIAP-Aurora-Siger>
- Repositório original da equipe (Fase 3): <https://github.com/Gcarmonapy7/fiap-aurora-siger-fase3>